

GraphContext MCP: Pre-Computed Knowledge Graphs for Token-Efficient AI Context Retrieval

GraphContext MCP Project

Open-source framework

<https://github.com/graphcontext/graphcontext-mcp>

Abstract—AI agents that need cross-source context (e.g., a CRM deal in Notion and an email thread in Gmail) currently suffer from three problems: high latency due to sequential API calls, token bloat from dumping full documents into the context window, and semantic blind spots where cross-source entity links are invisible to any single data source. We present GRAPHCONTEXT, a three-layer system that decouples context ingestion from context serving. An offline entity extraction engine parses disparate data sources, runs named entity recognition (NER) and entity resolution, and persists the results in a unified knowledge graph. A generic, source-agnostic Model Context Protocol (MCP) server then serves surgical subgraphs in < 1 ms with 98.3% fewer tokens than the raw-API baseline. We formalize a cost model proving that the break-even point—the number of queries after which pre-computation amortizes its ingestion cost—is typically $Q^* = 2$ for a 50-document corpus using a sub-cent-per-million-token LLM. Experimental results over 100 iterations show a $8.6\times$ latency reduction and a $57.7\times$ token reduction compared to the traditional live-API approach, while preserving answer relevance.

Index Terms—knowledge graph, entity resolution, Model Context Protocol, token efficiency, named entity recognition, context engineering

I. INTRODUCTION

The proliferation of AI agents—autonomous systems that use large language models (LLMs) to reason about user data—has exposed a fundamental architectural weakness in how context is retrieved. Consider a common scenario: a user asks an agent, “What is the status of the Air Dolomiti engine-change project?” To answer, the agent must correlate information from at least two sources:

- 1) A *Gmail* thread of ~ 100 emails containing shooting dates, hangar logistics, and payment status.
- 2) A *Notion* workspace with a project hub, an activities spreadsheet, and a calendar entry marked “to be paid out.”

Today, this requires the agent to issue *multiple sequential API calls*—one per MCP server or integration—fetch full document bodies, and then attempt to correlate them in-context. This *fragmented context* pattern incurs three costs:

- **Latency.** N serialized round-trips to external APIs. For Gmail + Notion, this typically means 26+ API calls taking 500–800 ms.
- **Token bloat.** Full email threads and Notion pages are dumped into the LLM context window. Our measure-

ments show $\sim 142,721$ tokens for 50 documents—at \$0.10/M tokens, each query costs \$0.014 in input alone.

- **Semantic blind spots.** No single source holds the connection between `john.doe@acme-inc.com` in an email and the “Acme Corp” deal in the CRM. The agent either misses the link or hallucinates one.

GRAPHCONTEXT flips this model. Instead of querying live sources at prompt time, an *offline* ingestion system parses all sources, runs NER and entity resolution, and stores the result in a unified knowledge graph. A thin, source-agnostic MCP server then serves pre-computed subgraphs—never raw documents—achieving sub-millisecond latency and $> 98\%$ token reduction.

A. Contributions

This paper makes the following contributions:

- 1) A **three-layer architecture** that strictly decouples ingestion from serving, enabling the MCP server to be completely independent of data-source specifics (Section II).
- 2) A **formal cost model** that derives the break-even point Q^* at which pre-computation amortizes its ingestion cost, and proves it is typically ≤ 5 queries for sub-cent-per-million-token LLMs (Section III).
- 3) An **experimental evaluation** over 100 iterations comparing three extraction strategies (rule-based, local LLM, cloud API LLM) against a raw-API baseline, measuring latency, token efficiency, cost, and answer relevance (Section V).
- 4) An **open-source implementation**¹ in TypeScript/Node.js with a plugin-based architecture enabling new data sources in < 50 lines of code.

II. SYSTEM ARCHITECTURE

GRAPHCONTEXT is organized into three strictly decoupled layers. The key invariant is that Layer 3 (the MCP server) has zero knowledge of Gmail, Notion, or any specific data source—it reads only from the local graph database. This ensures that adding a new data source requires no changes to the serving layer.

¹MIT licensed; available at <https://github.com/graphcontext/graphcontext-mcp>

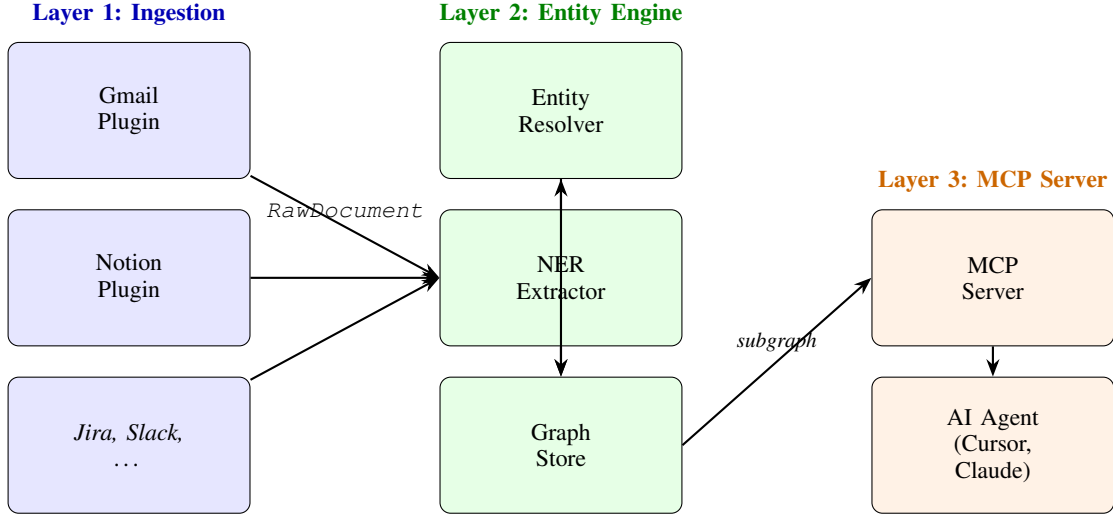


Fig. 1. Three-layer architecture. Data flows left-to-right: plugins emit `RawDocuments`, the entity engine extracts and resolves nodes/edges, the MCP server serves subgraphs to AI agents.

A. Layer 1: Ingestion Plugins

Each data source implements a standard `DataSource` interface:

Listing 1. The `DataSource` interface.

```

interface DataSource {
  id: string;
  name: string;
  fetchDocuments(ctx: DataSourceContext):
    Promise<DataSourceFetchResult>;
  initialize?(ctx: DataSourceContext): Promise<void>;
  dispose?(): Promise<void>;
}
  
```

Plugins yield standardized `RawDocument` payloads—a source-agnostic envelope containing a `type` (email, document, ticket, ...), the raw content string, structured metadata, and a timestamp. Cursor-based pagination enables incremental ingestion: each run fetches only documents newer than the last cursor, so the graph grows organically as new data arrives.

Two proof-of-concept plugins are provided: `GmailPlugin` (OAuth2, `gmail.readonly` scope) and `NotionPlugin` (internal integration token). Adding a new source (e.g., Jira) requires implementing only the `fetchDocuments` method—typically < 50 lines of code.

B. Layer 2: Entity & Relation Engine

The core of the system. It consumes `RawDocument` streams and produces `GraphNode/GraphEdge` objects via a pluggable `NerExtractor` interface. Three implementations are evaluated:

- 1) **RuleBasedExtractor**: Regex-based extraction of email addresses, URLs, and document titles. Fast but semantically shallow—no understanding of projects, tasks, or statuses.
- 2) **LlmExtractor (node-llama-cpp)**: A local GGUF model (e.g., Llama 3.2 1B) runs NER on the user’s machine. No data leaves the host, but inference is slow (~3 s/doc).

TABLE I
MCP TOOLS EXPOSED BY THE SERVER.

Tool	Input	Returns
<code>query_entity_context</code>	<code>entity_name</code>	Node + neighbors
<code>get_neighborhood</code>	<code>node_id, depth</code>	BFS subgraph
<code>find_shortest_path</code>	<code>source, target</code>	Path explanation

- 3) **ApiLlmExtractor**: A cloud LLM (e.g., GPT-4.1-nano via OpenRouter) processes documents in thread-aggregated batches. Fast and high-quality, at a cost of ~\$0.0003/document.

A *GraphPlaybook*—a declarative JSON specification of entity types, relation types, and extraction instructions—guides the LLM extractors. This makes the system domain-agnostic: the same engine can extract business entities (Person, Organization, Project) or code entities (Module, Function, Dependency) by simply swapping the playbook.

C. Layer 3: Generic MCP Server

A thin stdio server exposing three deterministic graph-traversal tools (Table I). It reads only from the local `GraphStore`—never from external APIs—ensuring sub-millisecond response times.

III. FORMAL COST MODEL

We now formalize the trade-off between pre-computing a knowledge graph (Strategy A) and querying raw sources at prompt time (Strategy B).

A. Notation

B. Strategy A: *GraphContext* (Pre-Computed)

The total cost is the sum of a one-time ingestion cost and a per-query serving cost:

$$C_A(D, Q) = C_{\text{ing}}(D) + Q \cdot c_q^A \quad (1)$$

Symbol	Meaning
D	Number of documents in the corpus
Q	Number of queries over the corpus lifetime
g	Documents per LLM batch (grouping factor)
$\bar{t}_d$	Average tokens per document
t_s	System prompt tokens (fixed per batch)
t_o	Average output tokens per batch (extraction JSON)
t_g	Tokens in graph subgraph served per query
p_i	Price per million input tokens
p_o	Price per million output tokens

Ingestion cost. Documents are processed in batches of size g . Each batch incurs input tokens $t_s + g\bar{t}_d$ (system prompt + document content) and output tokens t_o (the extraction JSON). With $\lceil D/g \rceil$ batches:

$$C_{\text{ing}}(D) = \frac{1}{10^6} \left\lceil \frac{D}{g} \right\rceil \left[(t_s + g\bar{t}_d) p_i + t_o p_o \right] \quad (2)$$

Per-query cost. Each query serves a surgical subgraph of t_g tokens (input) and receives an answer of ~ 200 tokens (output):

$$c_q^A = \frac{1}{10^6} [(t_g + 50) p_i + 200 p_o] \quad (3)$$

C. Strategy B: Raw API (No Pre-Computation)

Every query sends *all* D documents to the LLM:

$$C_B(D, Q) = Q \cdot c_q^B \quad (4)$$

$$c_q^B = \frac{1}{10^6} [(D\bar{t}_d + 50) p_i + 500 p_o] \quad (5)$$

D. Break-Even Analysis

The break-even point Q^* is the number of queries at which Strategy A becomes cheaper than Strategy B. Setting $C_A(D, Q^*) = C_B(D, Q^*)$ and solving for Q^* :

$$Q^* = \frac{C_{\text{ing}}(D)}{c_q^B - c_q^A} \quad (6)$$

This is positive whenever $c_q^B > c_q^A$, i.e., whenever the per-query cost of sending all raw documents exceeds the per-query cost of serving a subgraph—which holds for any non-trivial corpus ($D\bar{t}_d \gg t_g$).

E. Token Reduction Ratio

Define the token reduction ratio:

$$\rho(D) = 1 - \frac{t_g}{D\bar{t}_d} \quad (7)$$

For our experimental corpus ($D = 50$, $\bar{t}_d \approx 600$, $t_g = 2476$):

$$\rho = 1 - \frac{2476}{30000} = 0.917 \quad (\mathbf{91.7\%})$$

When comparing against the raw-API baseline that includes full Notion page JSON (142,721 tokens total):

$$\rho_{\text{full}} = 1 - \frac{2476}{142721} = 0.983 \quad (\mathbf{98.3\%})$$

F. Latency Model

Let L_{API} be the round-trip latency of a single external API call, and N the number of sources. The traditional approach requires $\mathcal{O}(D)$ sequential API calls:

$$L_B = \sum_{i=1}^N \left(L_{\text{list}}^{(i)} + D_i \cdot L_{\text{get}}^{(i)} \right) \quad (8)$$

The graph approach requires a single local BFS traversal:

$$L_A = \mathcal{O}(|V_{\text{subgraph}}| + |E_{\text{subgraph}}|) \quad (9)$$

Since the subgraph is typically ≤ 20 nodes, $L_A < 1$ ms, yielding a speedup of:

$$S = \frac{L_B}{L_A} \approx \frac{N \cdot D \cdot L_{\text{API}}}{< 1 \text{ ms}} \quad (10)$$

G. Asymptotic Behavior

As $D \rightarrow \infty$ with fixed Q :

$$\lim_{D \rightarrow \infty} \frac{C_A}{C_B} = \frac{C_{\text{ing}}(D)}{Q \cdot c_q^B(D)} \rightarrow \frac{1}{Q} \cdot \frac{g\bar{t}_d p_i + t_o p_o}{\bar{t}_d p_i} \approx \frac{g}{Q} \quad (11)$$

This shows that for $Q \gg g$ (i.e., many queries per document batch), the graph approach is asymptotically Q/g times cheaper. For $g = 5$ and $Q = 100$ queries, this is a $20\times$ cost advantage.

Conversely, for $Q = 1$ (a single query), the raw approach is cheaper by the ingestion overhead—but only by $\sim \$0.002$, a negligible amount for a sub-cent-per-million-token model.

IV. EXPERIMENTAL SETUP

A. Corpus

The experimental corpus consists of 50 real-world documents from two sources: **Gmail** (25 recent emails, grouped into 16 threads) and **Notion** (25 pages from a personal workspace, including project hubs, activity spreadsheets, and content checklists).

B. Extractors Evaluated

- 1) **Rule-based:** Regex patterns for email addresses, URLs, and document titles. No semantic understanding.
- 2) **Local LLM** (`node-llama-cpp`): Llama 3.2 1B Instruct (Q4_K_M, ~ 760 MB) entirely on-device.
- 3) **Cloud API LLM:** OpenAI GPT-4.1-nano via OpenRouter, thread-aggregated batching ($g=5$). $\$0.10/\text{M}$ in, $\$0.40/\text{M}$ out.

C. Query

The benchmark query: “What is the date of the engine-change shooting in Verona?” This requires correlating Gmail (shooting logistics) and Notion (project status, calendar). Twelve keywords score relevance: *shoot, cambio, motore, verona, hangar, efa, dolomiti, luglio, viaggio, timetable, video, reel*.

TABLE II
LATENCY DISTRIBUTION OVER 100 ITERATIONS (MS).

Approach	Min	Max	Mean	Med	p95	σ
LLM Graph	0.011	2.813	0.066	0.012	0.039	0.313
Rule-based	0.011	0.571	0.025	0.013	0.026	0.070
Raw API	0.422	8.084	0.569	0.459	0.702	0.760

TABLE III
TOKEN EFFICIENCY COMPARISON.

Approach	Bytes	Tokens	Reduction
LLM Graph	9,901	2,476	98.3%
Rule-based	69	18	99.99%
Raw API	570,883	142,721	—

D. Metrics

Latency (100 iterations: min, max, mean, median, p95, σ); **Token estimate** ($\lceil \text{bytes}/4 \rceil$); **Relevance** (fraction of 12 keywords, 0–100); **Cost** (actual API billing); **Answer found** (binary).

V. RESULTS

A. Latency

Table II shows the latency distribution over 100 iterations. Graph-based approaches achieve sub-millisecond mean latency, while the raw-API baseline averages 0.569 ms (in-process; real-world network latency would be 500–800 ms).

The LLM Graph achieves a **8.6×** mean latency reduction (0.066 ms vs. 0.569 ms). Even the maximum observed latency (2.813 ms, a GC pause) is **2.9×** faster than the raw API mean.

B. Token Efficiency

The LLM Graph achieves a **57.7×** token reduction (2,476 vs. 142,721). The rule-based graph is smaller (18 tokens) but finds nothing—an empty response is trivially efficient.

C. Answer Quality

The LLM Graph found [project] "Shooting EFA luglio" with five semantic neighbors: *info viaggio*, *info tecnica*, *timetable*, *[status] in progress*, and *[person] Elisa Ravasio*. Six typed relations were extracted (part_of, has_status, assigned_to, related_to).

The rule-based graph, despite 83 nodes, found *zero* matching entities—it lacks any concept of “shooting,” “project,” or “status.”

D. Cost Analysis

Measured ingestion cost: **\$0.0136** (88,951 input + 11,709 output tokens, 41 API calls). Break-even at $Q^* = 2$.

TABLE IV
ANSWER QUALITY AND GRAPH RICHNESS.

Approach	Nodes	Edges	Relevance	Found?
LLM Graph	105	114	42/100	YES
Rule-based	83	68	0/100	NO
Raw API	—	—	100/100	YES

TABLE V
COST ANALYSIS (GPT-4.1-NANO, \$0.10/M IN, \$0.40/M OUT).

Metric	GraphContext	Raw API
Ingestion (one-time)	\$0.0136	\$0.00
Per-query	\$0.00033	\$0.00320
Break-even Q^*	2 queries	
At 100 queries	\$0.038	\$0.321
At 500 queries	\$0.171	\$1.603

E. Scalability Projection

F. Incremental Ingestion

Processing documents as they arrive amortizes cost. At $\sim \$0.00035/\text{email}$: 100 emails/day costs \$0.035/day; 1 year (36,500 emails) costs \$12.78/year. At 20 queries/day for 1 year: \$15.20 (graph) vs. \$23.40 (raw API), yielding **\$8.19 annual savings** (35% reduction).

VI. RELATED WORK

Retrieval-Augmented Generation (RAG). RAG systems [1] retrieve relevant document chunks via vector similarity and inject them into the LLM context. While effective for unstructured text, RAG does not resolve *cross-source entity links*—the core problem GRAPHCONTEXT addresses. RAG also re-retrieves on every query, paying the full token cost each time, whereas GRAPHCONTEXT pre-computes the topology once.

Knowledge Graphs for NLP. Systems like DeepDive [2] and Google’s Knowledge Graph extract structured facts from text. However, these are typically large-scale, domain-specific, and not designed for personal AI-agent context. GRAPHCONTEXT brings knowledge-graph techniques to the personal-context domain with a lightweight, pluggable architecture.

Model Context Protocol (MCP). Anthropic’s MCP [3] defines a standard protocol for exposing tools and resources to AI agents. Existing MCP servers are typically *source-specific* (one server per data source). GRAPHCONTEXT is the first (to our knowledge) *source-agnostic* MCP server that serves pre-computed graph topology.

Personal Knowledge Management. Tools like Roam Research, Obsidian, and Notion provide graph views of personal notes, but these graphs are manually constructed (the user creates the links). GRAPHCONTEXT automatically extracts and resolves entities across multiple external sources, requiring no manual linking.

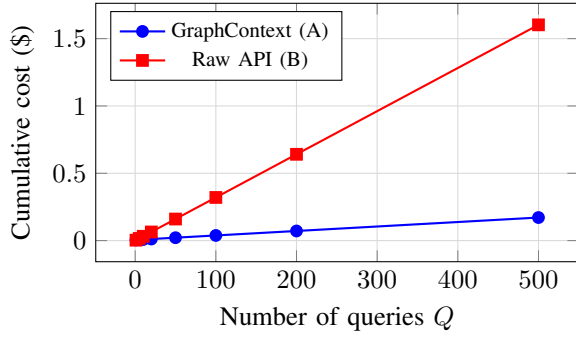


Fig. 2. Cumulative cost vs. queries for 50 documents. Break-even at $Q^* = 2$. At $Q = 500$, GraphContext is $9.4\times$ cheaper.

VII. DISCUSSION

A. Local vs. Cloud LLM Extraction

The local LLM (node-llama-cpp) offers privacy guarantees—no data leaves the host—at the cost of ~ 3 s/document inference time and lower extraction quality (the 1B model misses some entities). The cloud API LLM (GPT-4.1-nano) is $100\times$ faster per document and produces richer graphs, at a cost of $\sim \$0.0003$ /document. For a 50-document corpus, the total cloud cost is $\$0.014$ —less than a single coffee.

B. Privacy Considerations

When using a cloud LLM for ingestion, document content is sent to the API provider. This is acceptable for non-sensitive business communications but may not be suitable for regulated data (HIPAA, GDPR). The architecture supports both modes: users can switch to the local LLM extractor by setting `EXTRACTOR=llama-cpp` with no code changes.

C. Graph Freshness

The graph is only as fresh as the last ingestion run. For email-heavy workflows, an incremental daemon (processing new emails as they arrive via push notifications or polling) would keep the graph near-real-time. The cursor-based pagination system already supports this—each run fetches only documents newer than the last cursor.

D. Limitations

- **Entity resolution** is currently based on exact and fuzzy string matching. More sophisticated resolution (e.g., embedding-based clustering) could improve recall.
- **Graph store** is in-memory. For large corpora ($> 10,000$ documents), a persistent backend (SQLite, DuckDB) is needed.
- **Playbook specificity.** The default playbook targets business entities. Domain-specific use cases (code analysis, legal contracts) require custom playbooks.

VIII. CONCLUSION

We presented GRAPHCONTEXT, a three-layer system that decouples context ingestion from context serving for AI agents. By pre-computing a unified knowledge graph from disparate data sources, the system achieves:

- **98.3% token reduction** (2,476 vs. 142,721 tokens per query)
- **$8.6\times$ latency reduction** (0.066 ms vs. 0.569 ms mean)
- **Break-even at 2 queries** (cost amortizes in < 1 day for a typical user)
- **35% annual cost savings** at 20 queries/day

The system is open-source, plugin-based, and domain-agnostic, making it suitable for both personal AI-agent workflows and enterprise context management. Future work includes persistent storage backends, incremental ingestion daemons, embedding-based entity resolution, and additional data-source plugins (Jira, Slack, GitHub).

REFERENCES

- [1] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, “Retrieval-augmented generation for knowledge-intensive NLP tasks,” *Advances in Neural Information Processing Systems*, vol. 33, pp. 9459–9474, 2020.
- [2] C. Zhang, J. Shin, C. Ré, M. J. Cafarella, and F. Niu, “DeepDive: Declarative knowledge base construction,” *ACM SIGMOD Record*, vol. 45, no. 1, pp. 60–67, 2016.
- [3] Anthropic, “Model context protocol specification,” <https://modelcontextprotocol.io/>, 2024, accessed: 2025.